

Development · UAT · Production

Common plan for **all projects** — merge, deployment, maintain, rollback & versioning

Scope: **Frontend · API · Database**

Applies to: All web projects (e.g. CCSHAU, Siyana Website, future apps)

Document version: 2.0 · 8 September 2026

Environments: Development · UAT · Production

Layers: Frontend + API + Database

Table of Contents

1. Purpose & golden rules
2. Environment model (all projects)
3. Architecture diagrams
4. Versioning standard
5. Common merge steps
6. Common deployment steps
7. Common maintain steps
8. Common rollback steps
9. Frontend / API / Database checklist
10. Release & incident checklists
11. How to apply this to any project

1. Purpose & golden rules

This document is the **standard operating plan** for every Siyana Info Solutions web project. Use the same Development → UAT → Production flow, the same merge rules, and the same rollback approach so teams can maintain code safely and consistently.

Golden rule: Code in Git. Secrets outside Git (hosting + DB portals). One release tag = Frontend + API + Database migrations together. Always keep a rollback target.

Do

- Use 3 environments
- Promote via Pull Request
- Tag Production releases
- Record rollback target

Don't

- Commit secrets
- Push direct to Production
- Share Production DB with Dev
- Edit old migrations

Scope

- Frontend app
- API / server routes
- Database schema & data safety
- Not auth-policy extras

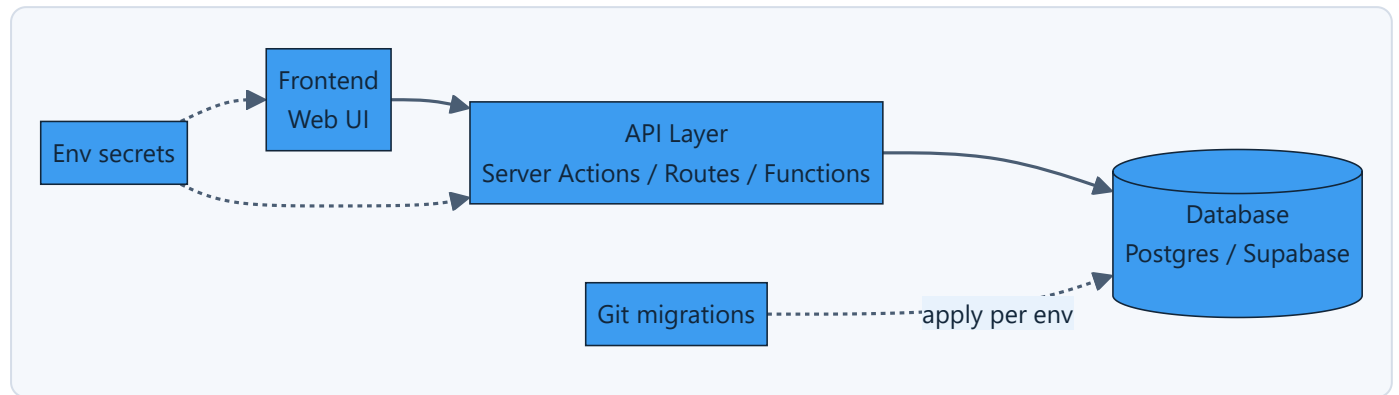
2. Environment model (all projects)

Environment	Git branch	Frontend	API	Database	Who uses
Development	develop / feature/*	Local + Dev/Preview host	Same app / services as Dev	Dev DB project	Developers
UAT	uat	UAT URL / domain	UAT deploy	UAT DB project	QA + stakeholders
Production	main + tag vX.Y.Z	Live domain	Live deploy	Production DB	End users

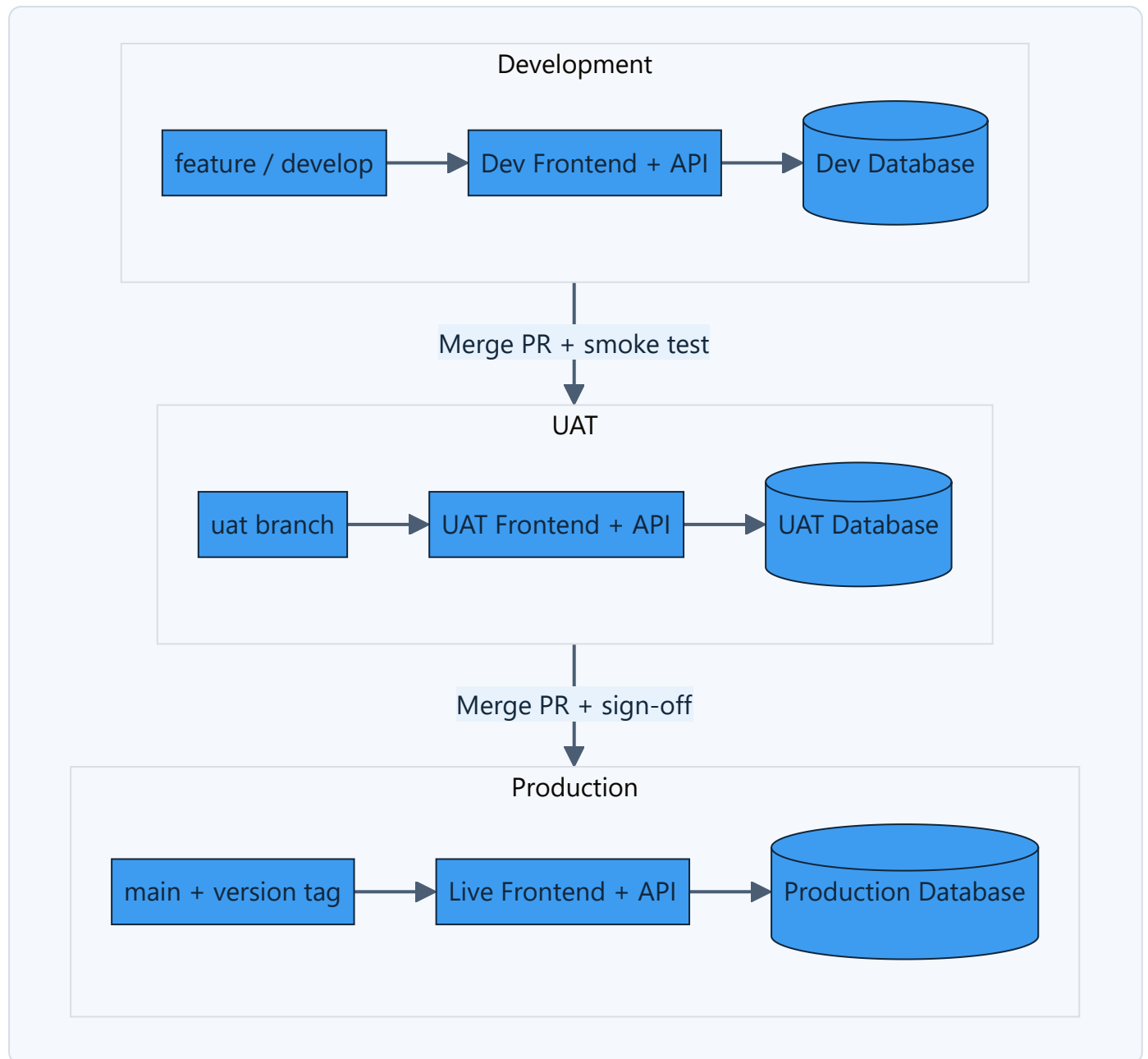
Never point Development or UAT frontends at the Production database. Each environment must have its own DB project (or clearly separated instance) and its own secret set.

3. Architecture diagrams

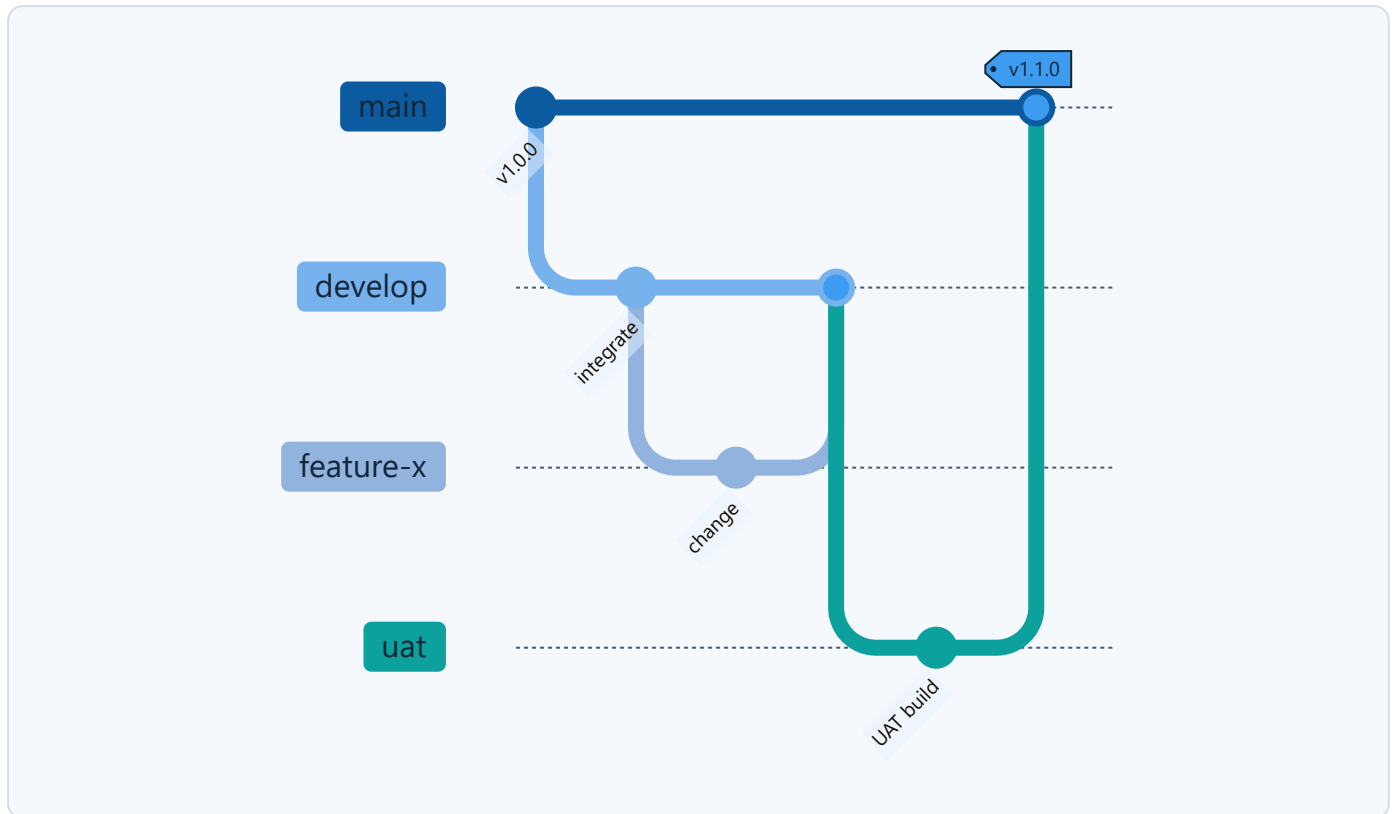
3.1 Standard stack layers



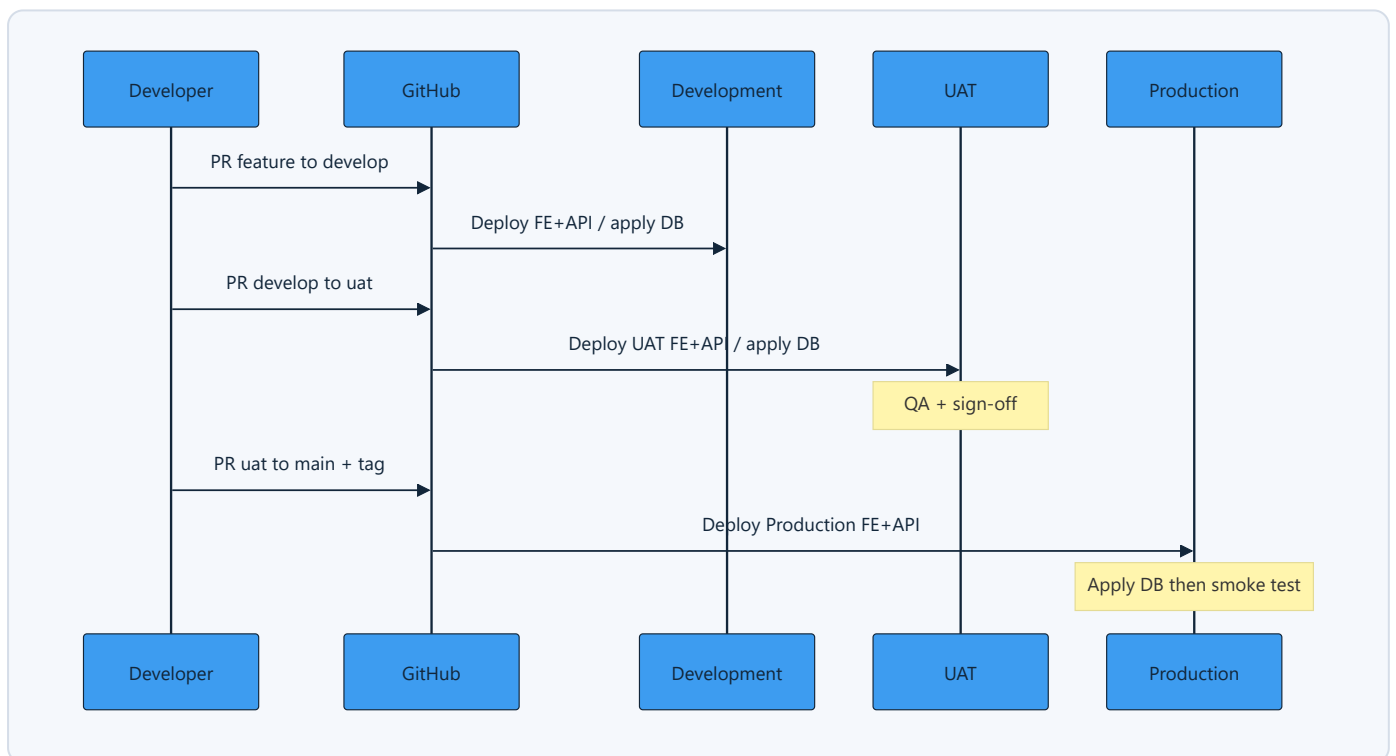
3.2 Three isolated environments



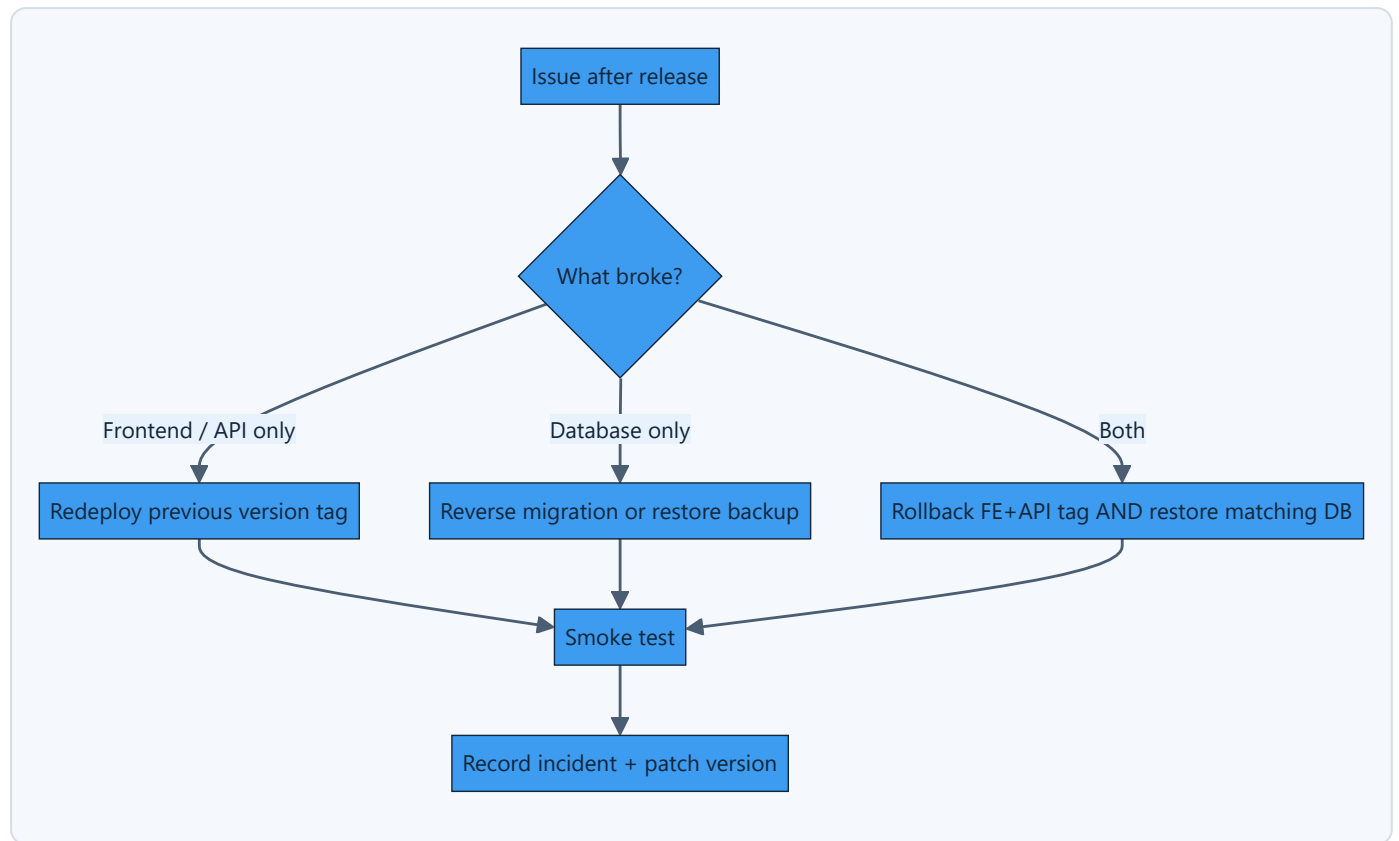
3.3 Common merge flow



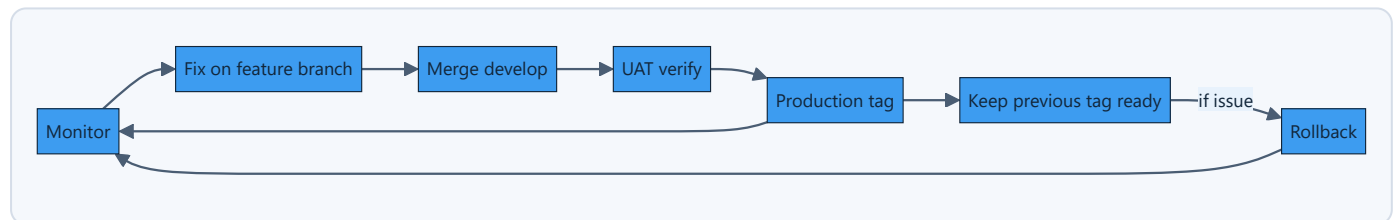
3.4 Common deployment pipeline



3.5 Rollback decision flow



3.6 Maintain cycle



4. Versioning standard

`vMAJOR.MINOR.PATCH`

MAJOR

= breaking Frontend / API / Database contract

MINOR

= new feature

PATCH

= bug fix / hotfix / rollback patch

Release artifact	Must include
Git tag <code>vX.Y.Z</code>	Exact commit for Frontend + API
Release notes	List of DB migrations in this version
Rollback target	Previous tag e.g. <code>vX.Y.Z-1</code>
Deploy record	Hosting deployment id / time

Version package: Frontend code + API code (same commit) + migration list + rollback tag.
Never release only one layer without noting the others.

5. Common merge steps

- 1 Create `feature/ticket-name` from `develop`.
- 2 Implement Frontend and/or API changes; add a **new** DB migration file if schema changes.
- 3 Open Pull Request → `develop`. Wait for CI (lint/test/build) to pass. Get review if required by project.
- 4 Merge to `develop`. Deploy Development. Apply Dev migrations.
- 5 When ready for QA: Pull Request `develop` → `uat`. Merge. Deploy UAT. Apply UAT migrations.
- 6 After UAT sign-off: Pull Request `uat` → `main`. Merge. Create tag `vX.Y.Z`.
- 7 Hotfix only: branch from `main` → `fix` → PR to `main` (patch tag) → also merge back to `uat` and `develop`.

Branch protection (all projects)

- No direct commits to `uat` or `main`
- Require Pull Request
- Require green CI before merge
- Block force-push on protected branches
- Keep repository private; least-privilege access
- Never store secrets in Git; use `.env.example` placeholders only

6. Common deployment steps

6.1 Development

1. Merge to `develop`
2. Auto/manual deploy Frontend + API to Dev host
3. Run DB migrations on Dev database
4. Smoke test critical screens + one write API

6.2 UAT

1. Merge `develop` → `uat`
2. Deploy Frontend + API to UAT host
3. Apply same migrations to UAT database
4. QA executes test plan; collect sign-off

6.3 Production

1. Merge `uat` → `main`
2. Create and push tag `vX.Y.Z`
3. Note previous production tag as rollback target
4. Confirm Production secrets are set
5. Deploy Frontend + API to Production
6. Apply Production DB migrations (after backup / restore point noted)
7. Run Production smoke checklist

Deploy order tip: Prefer backward-compatible DB changes. Expand schema → deploy code → later contract. For breaking changes, use a short maintenance window and a clear rollback plan.

7. Common maintain steps

Weekly / ongoing

- Monitor errors / failed deploys
- Keep `develop` healthy
- Do not let UAT drift far from Production
- Apply security dependency patches via normal PR flow

Every release

- Update release notes + migration list
- Confirm backup exists before Prod DB change
- Verify env vars still correct after host changes
- Retire unused feature branches

Maintain code quality

- Small PRs; one purpose per feature branch
- API input validation on server side
- Database RLS / least privileges
- No service-role / admin keys in Frontend
- Log errors without dumping secrets or full PII where avoidable

8. Common rollback steps

If this fails	Rollback action	Then
Frontend / API only	Redeploy previous hosting deployment or redeploy previous git tag	Smoke test UI + API health
Database only	Add reverse migration or restore DB backup to pre-release point	Confirm schema matches running app version
Both	Rollback Frontend+API to previous tag and restore DB to matching version	Full smoke test; publish patch notes

Step-by-step rollback

- 1 Stop further Production merges.
- 2 Identify failing layer (Frontend / API / Database / both).
- 3 Rollback Frontend+API to last good tag (hosting UI or git revert + patch tag).
- 4 If DB changed: reverse migration if safe; otherwise restore backup.
- 5 Ensure app version and DB schema match.
- 6 Smoke test Production.
- 7 Open hotfix on `main` ; merge back to `uat` and `develop` .

After any DB restore, the running Frontend/API must be the version that matches that schema. Rolling only one side can leave Production broken.

9. Frontend / API / Database checklist

Layer	Merge	Deploy	Maintain	Rollback
Frontend	PR includes UI changes; CI build passes	Host build with correct <code>NEXT_PUBLIC_*</code> / public env per environment	Fix via feature PR; keep preview for review	Previous hosting deploy / previous tag
API	PR includes server routes/actions; auth checks intact	Server secrets set per env (service key, cron, webhooks)	Monitor 5xx; patch via PR	Same deploy unit as Frontend (redploy previous tag)
Database	New migration file in Git; never rewrite applied files	Apply Dev → UAT → Prod with same files	Backups on; review grants/RLS	Reverse migration or restore backup + match app tag

10. Release & incident checklists

Before UAT

- ☐ PR reviewed / CI green
- ☐ Migrations listed
- ☐ Dev smoke passed
- ☐ UAT secrets verified

Before Production

- ☐ UAT signed off
- ☐ Rollback target tag written in release notes
- ☐ Production backup / restore point noted
- ☐ Production env vars confirmed
- ☐ Tag `vX.Y.Z` created

After Production

- ☐ Health / home page OK
- ☐ One critical write path OK (form / admin / API)
- ☐ DB migration applied cleanly
- ☐ Release notes published

If rollback

- ☐ Layer identified
- ☐ FE/API restored to previous tag if needed
- ☐ DB reversed or restored if needed
- ☐ Versions matched
- ☐ Smoke passed
- ☐ Hotfix merged back to uat + develop

11. How to apply this to any project

Step	Action once per project
1	Create branches <code>develop</code> , <code>uat</code> , <code>main</code>
2	Protect <code>uat</code> and <code>main</code> (PR + CI)
3	Create 3 DB projects (Dev / UAT / Production)
4	Configure 3 hosting environments with separate secrets
5	Map branches to deploys
6	Start tagging Production as <code>vX.Y.Z</code>
7	Practice one rollback drill on UAT

Project mapping examples

Project	Frontend	API	Database
CCSHAU Website	Next.js <code>apps/web</code> on Vercel	Server Actions + <code>/api/*</code>	Supabase migrations <code>supabase/migrations</code>
Siyana Website	Next.js static export on Vercel	Supabase Edge Function + webhooks	Supabase lead tables + migrations
Any new project	App host (Vercel/other)	Server/API layer of that stack	Migration-based DB with backups

One-line summary for all projects: Develop on `develop` , validate on `uat` , release from `main` with version tags, deploy Frontend + API + Database together, and always keep a previous version ready to rollback.